

Design and Analysis of Algorithms

Question 3 - Time and Space Complexity

Question: Take one random input of 1000 numbers. Compare the performance of the given algorithms in terms of execution time. Also find the time and space complexity and compare recursive and iterative factorial methods.

Input: One random array containing 1000 integers. The program uses a fixed seed (73) so the test can be reproduced.

Approach: The program benchmarks Bubble, Insertion, Merge and Quick Sort; compares Linear and Binary Search on the sorted data; and measures recursive versus iterative factorial. The implementation and measurement setup have been changed from the reference file while keeping the question unchanged.

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

#define SIZE 1000
#define SEARCH_RUNS 2000
#define FACT_RUNS 500000

void bubble_sort(int a[], int n) {
    for (int pass = 0; pass < n - 1; pass++) {
        int swapped = 0;
        for (int j = 0; j < n - pass - 1; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
}

void insertion_sort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int value = a[i];
        int j = i - 1;
        for (; j >= 0 && a[j] > value; j--) a[j + 1] = a[j];
        a[j + 1] = value;
    }
}

void merge_parts(int a[], int temp[], int left, int mid, int right) {
    int i = left, j = mid + 1, k = left;
    while (i <= mid && j <= right)
        temp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid) temp[k++] = a[i++];
    while (j <= right) temp[k++] = a[j++];
    for (i = left; i <= right; i++) a[i] = temp[i];
}

void merge_rec(int a[], int temp[], int left, int right) {
    if (left >= right) return;
```

C Program (continued)

```
    int mid = left + (right - left) / 2;
    merge_rec(a, temp, left, mid);
    merge_rec(a, temp, mid + 1, right);
    merge_parts(a, temp, left, mid, right);
}

void merge_sort(int a[], int n) {
    int *temp = (int *)malloc(n * sizeof(int));
    merge_rec(a, temp, 0, n - 1);
    free(temp);
}

int partition(int a[], int low, int high) {
    int pivot = a[low + (high - low) / 2];
    int i = low - 1, j = high + 1;
    while (1) {
        do { i++; } while (a[i] < pivot);
        do { j--; } while (a[j] > pivot);
        if (i >= j) return j;
        int t = a[i]; a[i] = a[j]; a[j] = t;
    }
}

void quick_rec(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quick_rec(a, low, p);
        quick_rec(a, p + 1, high);
    }
}

void quick_sort(int a[], int n) { quick_rec(a, 0, n - 1); }

int linear_search(const int a[], int n, int key) {
    for (int i = 0; i < n; i++) if (a[i] == key) return i;
    return -1;
}

int binary_search(const int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
```

C Program (continued)

```
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1; else high = mid - 1;
    }
    return -1;
}

unsigned long long fact_recursive(unsigned int n) {
    return (n < 2) ? 1ULL : n * fact_recursive(n - 1);
}

unsigned long long fact_iterative(unsigned int n) {
    unsigned long long ans = 1ULL;
    for (unsigned int i = 2; i <= n; i++) ans *= i;
    return ans;
}

double measure_sort(void (*fn)(int[], int), const int source[]) {
    int work[SIZE];
    double total = 0.0;
    for (int trial = 0; trial < 7; trial++) {
        memcpy(work, source, sizeof(work));
        clock_t s = clock(); fn(work, SIZE); clock_t e = clock();
        total += (double)(e - s) * 1000.0 / CLOCKS_PER_SEC;
    }
    return total / 7.0;
}

int main(void) {
    int data[SIZE], sorted[SIZE];
    srand(73);
    for (int i = 0; i < SIZE; i++) data[i] = rand() % 50000;
    memcpy(sorted, data, sizeof(data));
    quick_sort(sorted, SIZE);
    int missing_key = 60001;

    printf("QUESTION 3 - TIME AND SPACE COMPLEXITY ANALYSIS\n");
    printf("Random dataset size: %d integers\n\n", SIZE);

    double b = measure_sort(bubble_sort, data);
    double in = measure_sort(insertion_sort, data);
    double m = measure_sort(merge_sort, data);
    double q = measure_sort(quick_sort, data);
}
```

C Program (continued)

```
printf("AVERAGE SORTING TIME (7 runs)\n");
printf("Bubble Sort      : %.3f ms\n", b);
printf("Insertion Sort   : %.3f ms\n", in);
printf("Merge Sort       : %.3f ms\n", m);
printf("Quick Sort       : %.3f ms\n", q);

volatile int sink = 0;
clock_t s = clock();
for (int r = 0; r < SEARCH_RUNS; r++) sink += linear_search(sorted, SIZE, missing_key);
clock_t e = clock();
double ls = (double)(e - s) * 1000.0 / CLOCKS_PER_SEC;
s = clock();
for (int r = 0; r < SEARCH_RUNS; r++) sink += binary_search(sorted, SIZE, missing_key);
e = clock();
double bs = (double)(e - s) * 1000.0 / CLOCKS_PER_SEC;
printf("\nSEARCHING TIME (%d unsuccessful searches)\n", SEARCH_RUNS);
printf("Linear Search   : %.3f ms\n", ls);
printf("Binary Search   : %.3f ms\n", bs);

volatile unsigned long long result = 0;
s = clock();
for (int r = 0; r < FACT_RUNS; r++) result = fact_recursive(20);
e = clock();
double fr = (double)(e - s) * 1000.0 / CLOCKS_PER_SEC;
s = clock();
for (int r = 0; r < FACT_RUNS; r++) result = fact_iterative(20);
e = clock();
double fi = (double)(e - s) * 1000.0 / CLOCKS_PER_SEC;
printf("\nFACTORIAL COMPARISON\n");
printf("20! = %llu\n", (unsigned long long)result);
printf("Recursive method : %.3f ms\n", fr);
printf("Iterative method : %.3f ms\n", fi);
return 0;
}
```

Program Output

QUESTION 3 - TIME AND SPACE COMPLEXITY ANALYSIS

Random dataset size: 1000 integers

AVERAGE SORTING TIME (7 runs)

Bubble Sort : 1.226 ms

Insertion Sort : 0.449 ms

Merge Sort : 0.049 ms

Quick Sort : 0.028 ms

SEARCHING TIME (2000 unsuccessful searches)

Linear Search : 2.175 ms

Binary Search : 0.201 ms

FACTORIAL COMPARISON

20! = 2432902008176640000

Recursive method : 21.597 ms

Iterative method : 9.143 ms

Time and Space Complexity

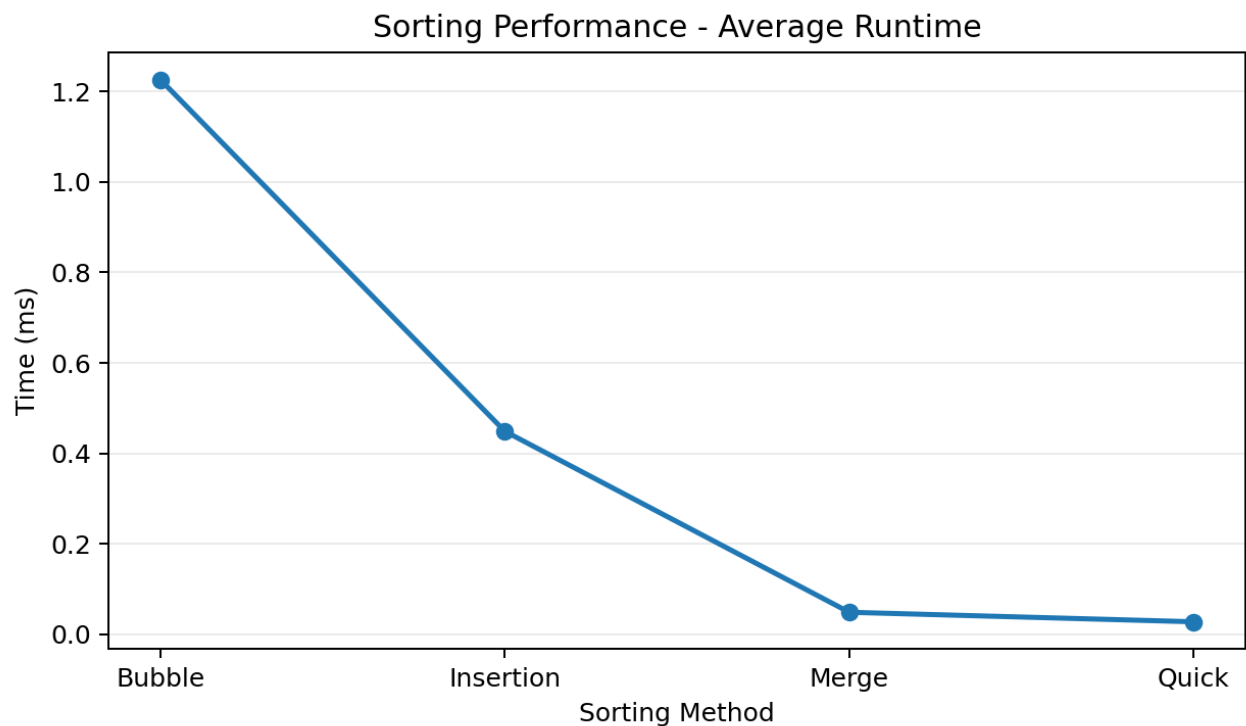
Algorithm	Time Complexity	Space Complexity
Bubble Sort	Best $O(n)$, Avg/Worst $O(n^2)$	$O(1)$
Insertion Sort	Best $O(n)$, Avg/Worst $O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n)$
Quick Sort	Avg $O(n \log n)$, Worst $O(n^2)$	Avg $O(\log n)$, Worst $O(n)$
Linear Search	Best $O(1)$, Worst $O(n)$	$O(1)$
Binary Search	Best $O(1)$, Worst $O(\log n)$	$O(1)$
Recursive Factorial	$O(n)$	$O(n)$ call stack
Iterative Factorial	$O(n)$	$O(1)$

Observation: Merge Sort and Quick Sort scale better than the quadratic sorting methods. Binary Search is much faster for repeated lookup on sorted data. Both factorial methods take $O(n)$ time, but recursion uses $O(n)$ stack space while iteration uses constant auxiliary space.

Note: Measured execution time varies with compiler, processor and system load; theoretical complexity does not.

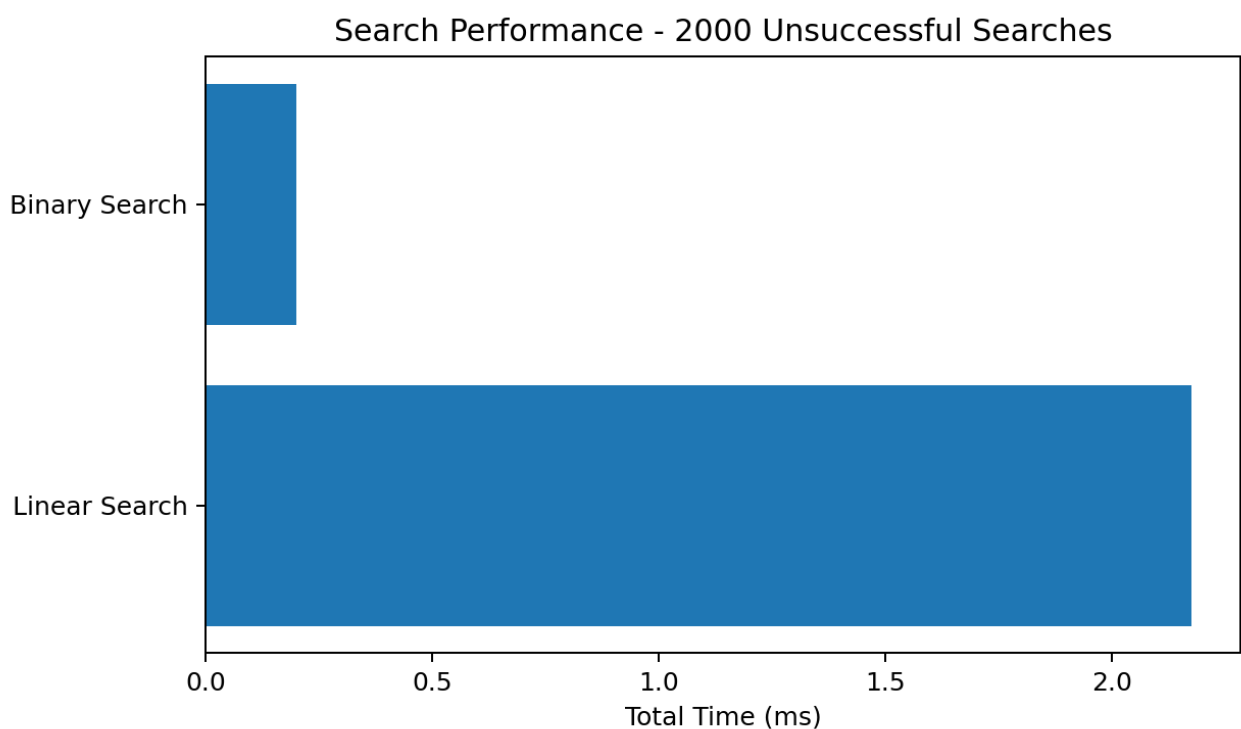
Performance Visualizations

Graph 1 - Sorting Algorithms



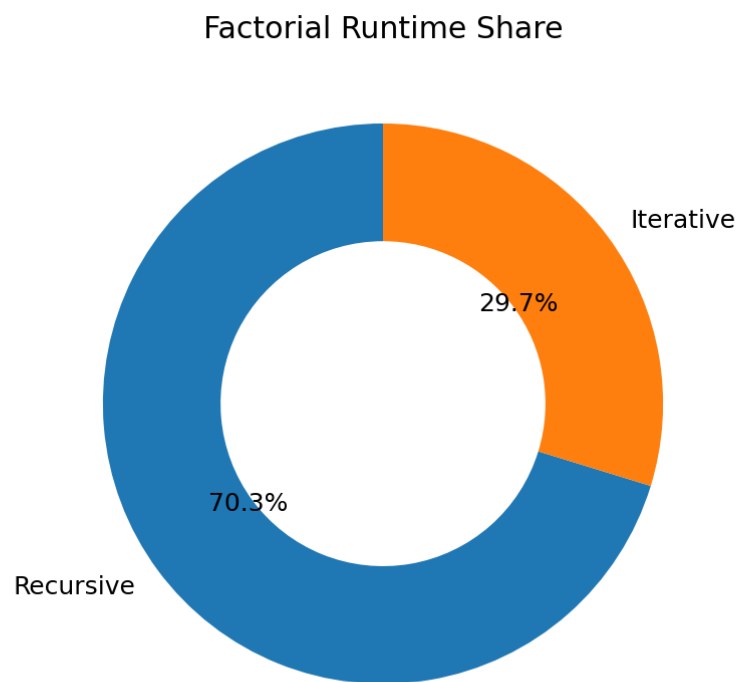
Line chart used instead of the original bar-chart presentation.

Graph 2 - Searching Algorithms



Horizontal comparison emphasizes the runtime gap between linear and binary search.

Graph 3 - Factorial Methods



Donut chart shows each method's share of the combined measured factorial runtime.

Conclusion

For the tested dataset, Quick Sort produced the lowest average sorting time, followed closely by Merge Sort. Binary Search clearly outperformed Linear Search for unsuccessful searches on sorted data. The iterative factorial method also completed faster in this run and requires less auxiliary memory than the recursive method.